
Claude Code 引き継ぎ

illustrator-scraperscraper

プロジェクト引き継ぎ書

Version 1.0

作成日: 2026年4月21日

プロジェクト: illustrator-scraperscraper

配置先: ~/roadie/projects/illustrator-scraperscraper/

発行: 株式会社Roadie

illustrator-scraper

イラストレーター候補者を X (旧 Twitter) から自動収集し、Googleスプレッドシートの候補プールに流し込む
半自動スクレイピングシステム

バージョン: 0.1.0 (初版)

作成日: 2026-04-21

オーナー: Roadie (Bisque ブランド運用)

関連ドキュメント: [Notion運用マニュアル v1.0](#)

プロジェクト概要

目的

BLサンド・Capuri・BerryFeel・Webtoon の各プロジェクトで依頼可能なイラストレーター候補を継続的に発掘するため、X (旧Twitter) のフォログラフを辿って候補者リストを自動生成する。

なぜ必要か

- 現在は Producer が手動で既存イラストレーターのフォロー欄を1つずつ確認しており、極めて非効率
- Notion DB はスクリーナー・オーナー・連絡担当が使うが、候補探索の初期段階では使わない (Notion は重いので軽量化したい)
- 候補探索 → スクリーナー判定 → Notion転記 の3段階を分離し、Notion に入るデータの純度を高める

アーキテクチャ全体像



本プロジェクトは上記フローの ①～④ を担当する。GAS部分は別プロジェクト（ `gas-illustrator-sync` ）で管理。

技術スタック

領域	技術	備考
言語	TypeScript 5.x	ESM形式
ランタイム	Node.js 20 LTS	
スクレイピング	Browser Use 2.0	既存ノウハウあり（DLsite/FANZA売上集計で使用）
Notion連携	@notionhq/client	マスターDBからシード抽出
スプレッドシート連携	googleapis (Google Sheets API v4)	候補プール書き込み
スケジューラー	node-cron	ローカル実行または GitHub Actions
ログ	pino + pino-pretty	JSON構造化ログ
テスト	vitest	統合テスト中心
パッケージマネージャ	pnpm	モノレポ構成の場合を考慮
Lint/Format	eslint + prettier	

技術選定の理由

Browser Use 2.0 を採用する理由

- Twitter API Basic は月額 \$200 と高額
- DLsite/FANZA売上集計で培った安定運用ノウハウを流用可能
- フォロー一覧ページは HTML で取得でき、API不要で実装可能

TypeScript ESM を採用する理由

- ユーザーの標準スタック（Next.js + TS）と整合
- 型安全でNotion APIの複雑なレスポンスを扱いやすい

ディレクトリ構造

```

~/roadie/projects/illustrator-scraper/
├── CLAUDE.md          # このファイル (Claude Code用指示書)
├── README.md          # 人間用ドキュメント
├── package.json
├── tsconfig.json
├── .env.example        # 環境変数テンプレート
├── .env               # 実運用の環境変数 (gitignore)
├── .gitignore
├── src/
│   ├── index.ts        # エントリーポイント (バッチ実行)
│   ├── config.ts       # 環境変数読み込み
│   ├── types.ts        # 型定義
│   ├── notion/
│   │   ├── client.ts   # Notion SDK ラッパー
│   │   └── seed-fetcher.ts # シードアカウント抽出
│   ├── scraper/
│   │   ├── browser.ts  # Browser Use 2.0 ラッパー
│   │   ├── follow-list.ts # フォロワー覧取得
│   │   └── profile.ts   # プロフィール取得
│   ├── filter/
│   │   ├── index.ts    # メインフィルタ
│   │   ├── profile-keyword.ts # プロフィールキーワード判定
│   │   ├── follower-count.ts # フォロワー数判定
│   │   ├── post-frequency.ts # 投稿頻度判定
│   │   └── duplicate.ts # 重複除外
│   ├── sheets/
│   │   ├── client.ts   # Google Sheets API ラッパー
│   │   └── writer.ts    # 候補プール書き込み
│   └── utils/
│       ├── logger.ts   # pino 設定
│       ├── rate-limit.ts # レート制限
│       └── retry.ts     # リトライロジック
├── scripts/
│   ├── seed-dry-run.ts  # シード抽出のみの動作確認
│   ├── filter-dry-run.ts # フィルタのみの動作確認
│   └── local-batch.ts   # ローカル実行用エントリ
├── tests/
│   ├── filter.test.ts
│   ├── notion.test.ts
│   └── sheets.test.ts
├── .github/
│   └── workflows/
│       └── scheduled-batch.yml # GitHub Actionsでの定期実行 (任意)
└── docs/
    ├── ARCHITECTURE.md # 詳細アーキテクチャ
    ├── OPERATIONS.md   # 運用手順
    └── TROUBLESHOOTING.md # 問題発生時の対処

```

環境変数

`.env.example` をテンプレートとして、各自で `.env` を作成する。

```
# === Notion ===
NOTION_API_KEY=secret_XXXXXXXXXXXX
NOTION_MASTER_DB_ID=1d5793bb-0629-4b6f-8a9a-3720c6a53139
# データソースURL: collection://1d5793bb-0629-4b6f-8a9a-3720c6a53139

# === Google Sheets ===
GOOGLE_SERVICE_ACCOUNT_EMAIL=xxx@xxx.iam.gserviceaccount.com
GOOGLE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n"
GOOGLE_SPREADSHEET_ID=1AbCdEfGhIjKlMnOpQrStUvWxYz
SCREENING_SHEET_NAME=候補プール

# === Browser Use 2.0 ===
# X のログイン情報（スクレイピング用の専用アカウントを推奨）
X_USERNAME=scrapper_account@example.com
X_PASSWORD=XXXXXXXXXXXX
X_SESSION_COOKIE=          # 任意：ログイン済みセッションを再利用

# === スクレイピング挙動 ===
SCRAPER_MAX_SEEDS_PER_RUN=10          # 1回のバッチで処理するシード数
SCRAPER_MAX_FOLLOWS_PER_SEED=200      # 1シードあたり取得するフォロー数上限
SCRAPER_DELAY_MIN_MS=3000             # リクエスト間隔の最小値
SCRAPER_DELAY_MAX_MS=8000             # リクエスト間隔の最大値
SCRAPER_HEADLESS=true                 # 本番はtrue、デバッグはfalse

# === フィルタ閾値 ===
FILTER_MIN_FOLLOWERS=500
FILTER_MAX_FOLLOWERS=50000
FILTER_MIN_POSTS_LAST_30D=3

# === ログレベル ===
LOG_LEVEL=info
```

認証情報の取得手順

Notion API Key 1. <https://www.notion.so/profile/integrations> にアクセス 2. 「New integration」→ workspace 選択 → 作成 3. Secret を `.env` の `NOTION_API_KEY` にコピー 4. マスターDBページ右上「...」→「接続の追加」で作成したインテグレーションを追加 ### Google Service Account 1. <https://console.cloud.google.com> でプロジェクト作成 2. IAM → Service Accounts → 作成 3. キーをJSON形式でダウンロード 4. JSON内の `client_email` と `private_key` を `.env` にコピー 5. Google Sheetsを作成し、`client_email` を「編集者」として共有

タスク分解（Phase別実装計画）

Phase 0: プロジェクトセットアップ（1日）

- [] Node.js 20+ と pnpm インストール確認
- [] `pnpm init` でプロジェクト初期化
- [] TypeScript (ESM), `tsconfig.json` 設定
- [] ESLint + Prettier 設定
- [] vitest セットアップ
- [] `.env.example` 作成
- [] 認証情報取得と `.env` 作成
- [] Notion API の疎通確認（マスターDBから1件取得できるか）
- [] Google Sheets API の疎通確認（テスト用シートに1行書けるか）

Phase 1: Notion シードアカウント抽出（1日）

目的: マスターDBから「依頼成功」「殿堂入り」ステータスのイラストレーターを取得し、X アカウント一覧を作る。

- [] `src/notion/client.ts` : Notion SDK初期化ラッパー
- [] `src/notion/seed-fetcher.ts` : マスターDBクエリ実装
- フィルタ条件: `マスターステータス IN (依頼成功, 殿堂入り)`
- 取得プロパティ: `作家名`, `Xアカウント`, `画力ランク`, `TL適性`
- [] Xアカウントが空のレコードは除外
- [] ランクS/AはPriority High、B以下はPriority Normalとして返す
- [] `scripts/seed-dry-run.ts` で動作確認

出力データ型:

```
interface SeedAccount {
  notionPageId: string;
  artistName: string;
  xUsername: string; // "@" は含まない
  powerRank: 'S' | 'A' | 'B' | 'C';
  priority: 'high' | 'normal';
}
```

Phase 2: Browser Use 2.0 でフォロワー一覧取得（3～5日）

目的: シードアカウントのフォロワー一覧を取得する。

- [] `src/scrapper/browser.ts` : Browser Use 2.0 の初期化ラッパー
- [] X ログインフロー実装
- セッションCookie保存・再利用（再ログインを減らす）

- CAPTCHA検出時はログを残して処理中断
- [] `src/scrapper/follow-list.ts` : 指定ユーザーのフォロワー一覧取得
- `https://x.com/{username}/following` にアクセス
- 無限スクロールで最大 `SCRAPER_MAX_FOLLOWS_PER_SEED` 件取得
- 各フォロワーから username, displayName, bio, followerCount を抽出
- [] リクエスト間に `SCRAPER_DELAY_MIN_MS` ~ `SCRAPER_DELAY_MAX_MS` のランダム遅延
- [] 凍結・レート制限検出時は指数バックオフでリトライ
- [] `src/scrapper/profile.ts` : プロフィール詳細取得（投稿数、Pixivリンク等）

⚠ Xスクレイピングの注意点

- Xは頻繁にDOM構造を変える。セレクトはできるだけrole/aria-labelベースで記述 - 深夜時間帯（日本時間 2:00~5:00）に実行するのが安全 - 1日あたり10アカウント程度を上限とする - IPがブロックされた場合のためにVPN/プロキシ選択肢を残しておく - スクレイピング用に専用のXアカウントを作成する（メインアカウントは使わない）

Phase 3: フィルタリング（2日）

目的: 取得したフォロワー一覧から、明らかにイラストレーターでないアカウントを除外する。

- [] `src/filter/profile-keyword.ts`
- bio に以下のキーワードを含む → 通過
 - 日本語: イラスト, 絵, 漫画, デザイン, 絵描き, お絵かき
 - 英語: illustration, artist, illustrator, drawing
- 除外キーワード: AI, 生成AI, stable diffusion, midjourney
- bio が空の場合は保留（スクリーナー判定に回す）
- [] `src/filter/follower-count.ts`
- `FILTER_MIN_FOLLOWERS` ~ `FILTER_MAX_FOLLOWERS` の範囲のみ通過
- [] `src/filter/post-frequency.ts`
- 直近30日で `FILTER_MIN_POSTS_LAST_30D` 件以上の投稿
- [] `src/filter/duplicate.ts`
- 既存マスターDBのXアカウントと突合
- スプレッドシート既存データとも突合
- 重複は除外（ただしフラグは立てて記録）
- [] `src/filter/index.ts` : 全フィルタを統合

出力データ型:


```
interface Candidate {
  username: string;
  displayName: string;
  bio: string;
  followerCount: number;
  followingCount: number;
  postsLast30d: number;
  pixivUrl?: string; // bioから抽出
  portfolioUrl?: string; // bioから抽出
  seedAccount: string; // 検出元シード
  detectedAt: Date;
  isDuplicateInMaster: boolean;
  filterScore: number; // 0-100の合格度スコア
}
```

Phase 4: Google Sheets への書き込み（1日）

目的: フィルタ通過した候補者をスプレッドシートに追記する。

- [] `src/sheets/client.ts` : Google Sheets API 初期化
- [] `src/sheets/writer.ts` : 候補プールシートに追記
- 列順: 検出日, 検出元, Xアカウント, 表示名, プロフィール, フォロワー数, Pixivリンク, ポートフォリオ, 既存DB重複, 判定, 画力ランク候補, コメント, 転記状態, 転記日時
- 判定 • 画力ランク候補 • コメント • 転記状態 • 転記日時 は空白で書き込み（スクリーナー・GAS が埋める）
- [] 書き込み前にスプレッドシート側の既存Xアカウント一覧を取得して重複回避
- [] バッチ書き込み（1回のAPI呼び出しで複数行）

Phase 5: バッチ実行とスケジューリング（1日）

目的: 定期実行できる形にまとめる。

- [] `src/index.ts` : 全フェーズをオーケストレーション
 1. Notionからシード取得
 2. 各シードに対してフォロワー一覧取得
 3. フィルタ適用
 4. スプレッドシート書き込み
 5. 結果サマリーをログ出力
- [] `scripts/local-batch.ts` : ローカル実行用エントリ
- [] GitHub Actions ワークフロー（任意）
- `.github/workflows/scheduled-batch.yml`
- cron: `0 17 * * 1`（毎週月曜 日本時間 2:00）
- シークレットは GitHub Secrets に登録

運用フロー

週次バッチ実行

1. **月曜深夜 2:00**: GitHub Actions または ローカル cron で自動実行
2. バッチが完了すると、スプレッドシート「候補プール」シートに最大200～500件の候補が追加される
3. Slack Webhook で完了通知（任意）

スクリーナー作業（翌日以降）

1. スプレッドシートを開き、新規追加された行を確認
2. 1件ずつ X プロフィール・Pixiv を確認
3. J列で ○ / 保留 / × を判定
4. ○判定の場合は K列に画力ランク候補、L列にコメント
5. 週末までに 50 件確認完了を目標
6. スプレッドシート内の「📌 Notion へ転記」ボタンを押下（GAS別プロジェクトで実装）

オーナー・連絡担当作業

詳細は [Notion運用マニュアル v1.0](#) を参照。

エラーハンドリング方針

致命的エラー（処理中断）

- X アカウントのログイン失敗 → Slack Webhook で即時通知
- Notion API 認証エラー → 処理中断、ログに詳細
- Google Sheets API クォータ超過 → 6時間後に再試行

回復可能エラー（ログして継続）

- 特定シードのフォロワー一覧取得失敗 → スキップして次のシードへ
- 特定候補のプロフィール取得失敗 → その候補をスキップ
- 一時的なネットワークエラー → 最大3回までリトライ

ロギング方針

```
// pino で構造化ログを出力
logger.info({
  phase: 'scraping',
  seed: 'example_account',
  candidatesFound: 42,
  durationMs: 12345
}, 'シードの処理完了');

logger.error({
  phase: 'scraping',
  seed: 'example_account',
  error: err.message,
  stack: err.stack
}, 'シード処理失敗');
```

開発時の注意事項

Claude Code を使う場合

1. このファイル（CLAUDE.md）を読んだから作業開始
2. Phase 単位で実装し、各Phase完了時に動作確認
3. 環境変数は `.env` に記載し、絶対にコミットしない
4. Browser Use 2.0 の操作は デバッグ時のみ `SCRAPER_HEADLESS=false` に
5. Notion API 呼び出しは dry-run モード（ `scripts/seed-dry-run.ts` ）で先に確認

コーディング規約

- コードコメントは日本語
- 変数名・関数名は英語
- 関数は単一責任原則で小さく保つ
- 非同期処理は `async/await`、`Promise.all` 活用
- エラーは独自例外クラスでラップ（ `NotionError` , `ScrapingError` 等）

テスト方針

- ユニットテスト: フィルタロジック中心
- 統合テスト: Notion API・Sheets API はモック化
- E2Eテスト: 月1回、実際のAPI・X に対して実行

トラブルシューティング

X のログインに失敗する

- 2FA設定を一時的にOFFにするか、TOTPを自動入力する処理を追加 - セッションCookieが古い場合は削除して再ログイン - IP ブロックの可能性 → VPN経由で接続

Notion API が 404 を返す

- インテグレーションがDBに接続されているか確認（DBページ → 接続の追加） -
`NOTION\_MASTER\_DB\_ID` の指定が正しいか確認 - プロパティ名が変更されていないか確認（Notion UI 上で確認）

Google Sheets が「permission denied」

- サービスアカウントのメールアドレスがスプレッドシートに共有されているか確認 - 共有権限が「編集者」になっているか確認

フィルタ通過率が異常に低い

- FILTER_MIN_FOLLOWERS を下げる（例: 500 → 200） - プロフィールキーワードリストを拡張 - 除外キーワードが厳しすぎないか確認

今後の拡張候補

- [] Pixiv からのスクレイピング（Pixiv API or スクレイピング）
- [] ポートフォリオサイト（Potofu, lit.link）からの情報取得
- [] 画像から AI 絵判定（検出が難しいため後回し）
- [] イラストレーター活動の変化検知（非アクティブ化、受注停止等）
- [] 依頼履歴と成果の連携分析（マスターDB ⇄ CAMBLIN案件DB）

関連プロジェクト

プロジェクト	役割
このプロジェクト (illustrator-scraper)	X スクレイピング → スプレッドシート投入
gas-illustrator-sync (別途作成)	スプレッドシート → Notion 転記 (GAS)
イラストレーター情報DB (Notion)	マスターデータ (候補～依頼成功まで)
CAMBLIN案件管理DB	依頼成功後の案件進行管理

改訂履歴

バージョン	日付	内容	担当
0.1.0	2026-04-21	初版作成	三村